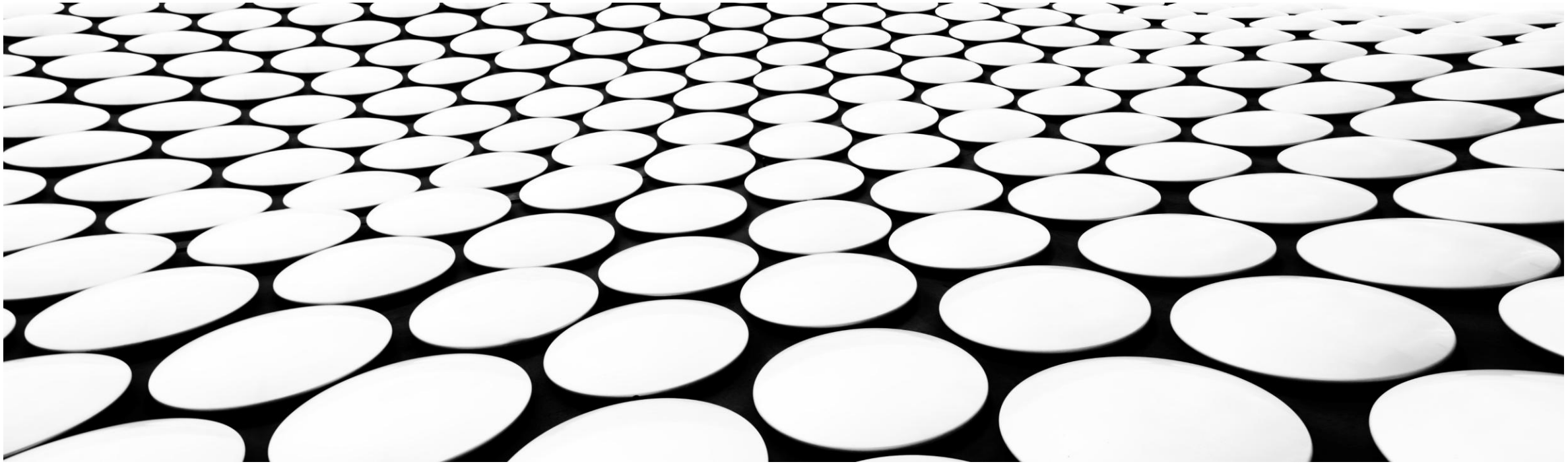
CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

TOPIC-7: **PROGRAMMING-II**

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA



These slides are based on the book: David Money Harris and Sarah L. Harris, Computer Organization and Design

BRANCHING

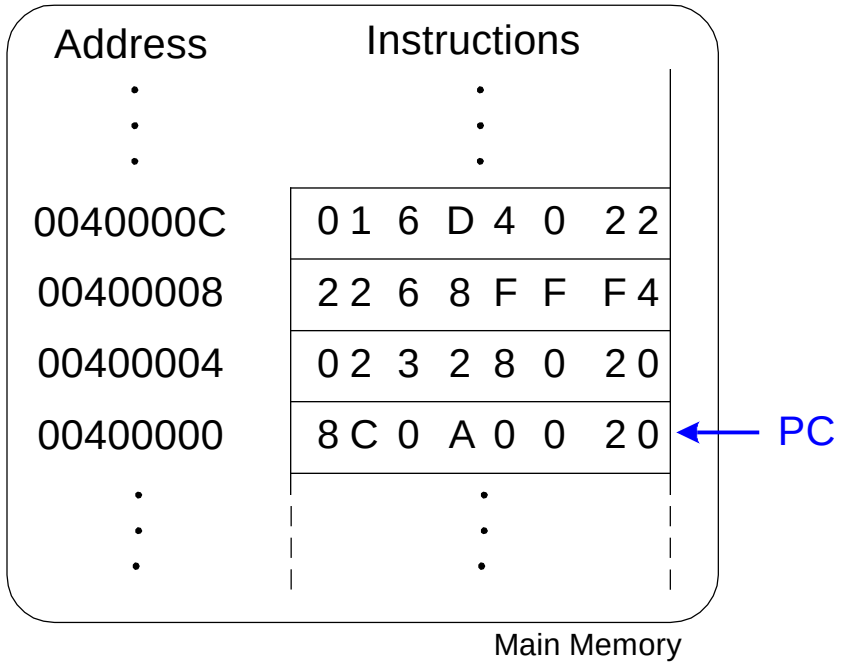
- Allows a program to execute instructions out of sequence.
- Types of branches:
  - **Conditional branches**
    - branch if equal (`beq`)
    - branch if not equal (`bne`)
  - **Unconditional branches**
    - jump (`j`)
    - jump register (`jr`)
    - jump and link (`jal`)

```
beq R1, R2 (2000)
```

REVIEW: THE STORED PROGRAM

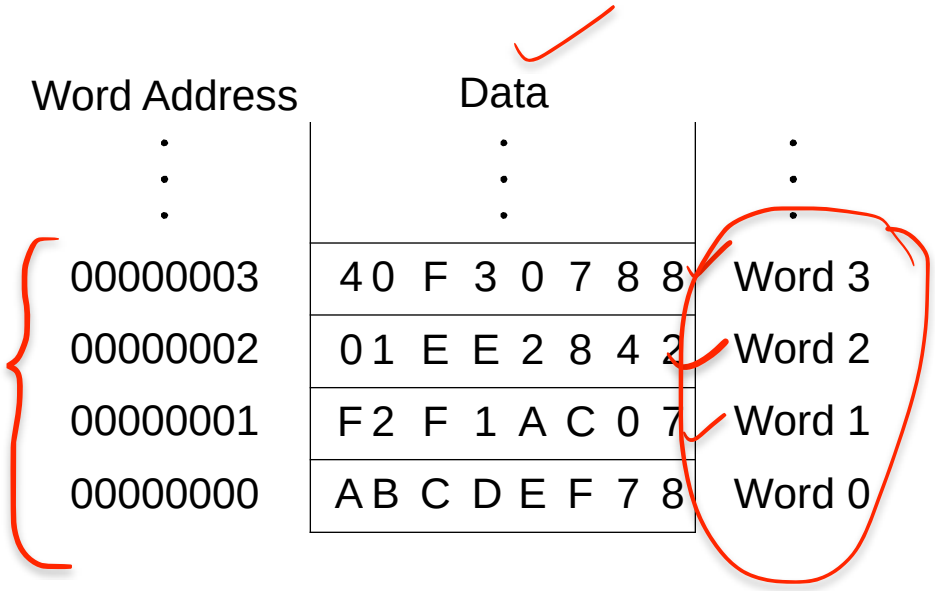
Assembly Code	Machine Code
`lw $t2, 32($0)`	`0x8C0A0020`
`add $s0, $s1, $s2`	`0x02328020`
`addi $t0, $s3, -12`	`0x2268FFF4`
`sub $t0, $t3, $t5`	`0x016D4022`

Stored Program



WORD-ADDRESSABLE MEMORY

- Each 32-bit data word has a unique address



- Data space in a cell: Word length of CPU
- Word: A unit of data that can be stored or operated
- A cell is uniquely identified by a binary number

READING WORD-ADDRESSABLE MEMORY

- Memory reads are called *loads*
- Mnemonic: *load word* (`lw`)
- **Example:** read a word of data at memory address 1 into `$s3`
- Memory address calculation:
  - add the *base address* (`$0`) to the *offset* (1)
  - address = ($0 + 1) = 1
- Any register may be used to store the base address.
- `$s3` holds the value 0xF2F1AC07 after the instruction completes.

Assembly code

```
lw $s3, 1($0)   # read memory word 1 into $s3
```

Word Address	Data	
⋮	⋮	⋮
00000003	40 F 30 7 8 8	Word 3
00000002	01 E E2 8 4 2	Word 2
00000001	F2 F 1A C 0 7	Word 1
00000000	AB C DE F 7 8	Word 0

WRITING WORD-ADDRESSABLE MEMORY

- Memory writes are called *stores*
- Mnemonic: *store word* (sw)
- **Example:** Write (store) the value held in $t4 into memory address 7
- Offset can be written in decimal (default) or hexadecimal
- Memory address calculation:
  - – add the base address ($0) to the offset (0x7)
  - – address: ($0 + 0x7) = 7
- Any register may be used to store the base address

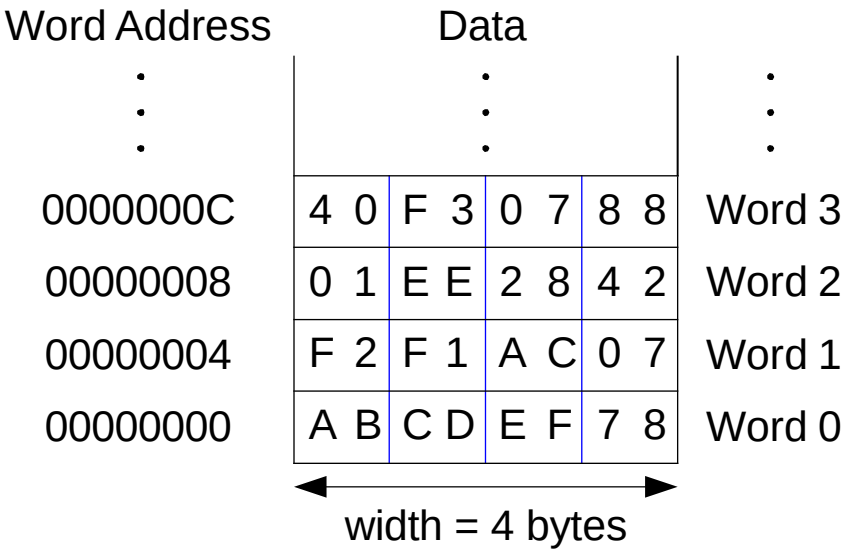
Assembly code

```
sw $t4, 0x7($0)   # write the value in $t4
                  # to memory word 7
```

Word Address	Data	
.	.	
.	.	
.	.	
.	.	
00000003	40 F 30 7 8 8	Word 3
00000002	01 E E2 8 4 2	Word 2
00000001	F2 F 1A C 0 7	Word 1
00000000	AB C D E F 7 8	Word 0

BYTE-ADDRESSABLE MEMORY

- Each data byte has a unique address
- Load/store words or single bytes: load byte (`lb`) and store byte (`sb`)
- Each 32-bit words has 4 bytes, so the word address increments by 4



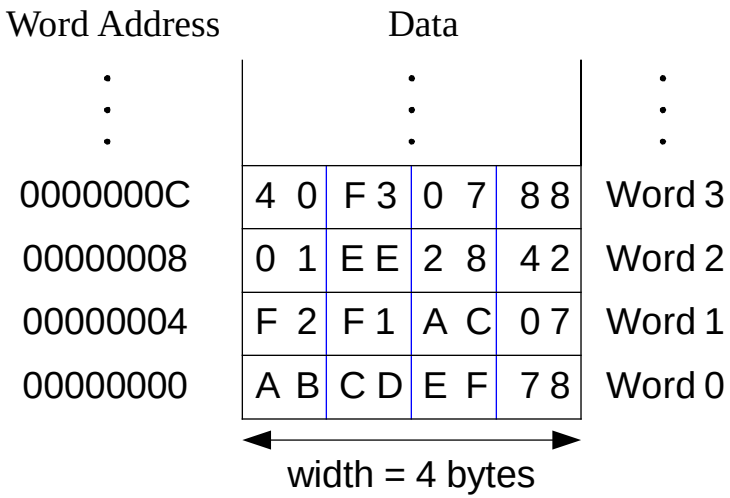
- Data space in a cell: 8bits = 1byte
- A cell is uniquely identified by a binary number

READING BYTE-ADDRESSABLE MEMORY

- The address of a memory word must now be multiplied by 4. For example,
  - the address of memory word 2 is 2 × 4 = 8
  - the address of memory word 10 is 10 × 4 = 40 (0x28)
- Load a word of data at memory address 4 into `$s3`.
- `$s3` holds the value 0xF2F1AC07 after the instruction completes.
- **MIPS is byte-addressed, not word-addressed**

MIPS assembly code

```
lw $s3, 4($0) # read word at address 4 into $s3
```

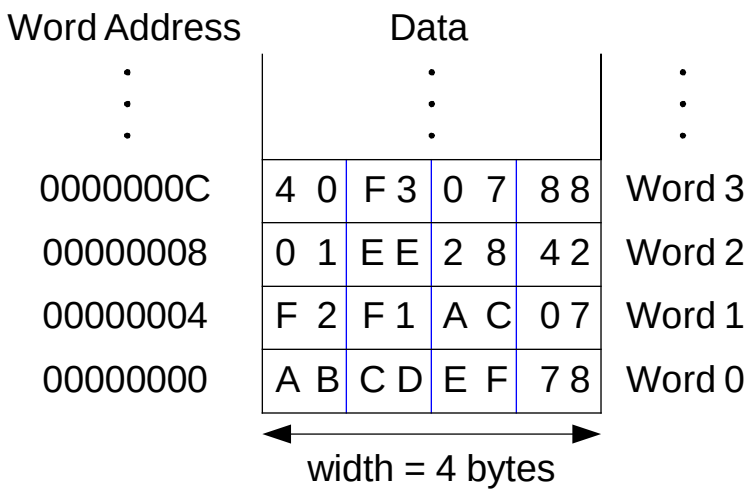


WRITING BYTE-ADDRESSABLE MEMORY

- **Example:** stores the value held in `$t7` into memory address 0x2C (44)

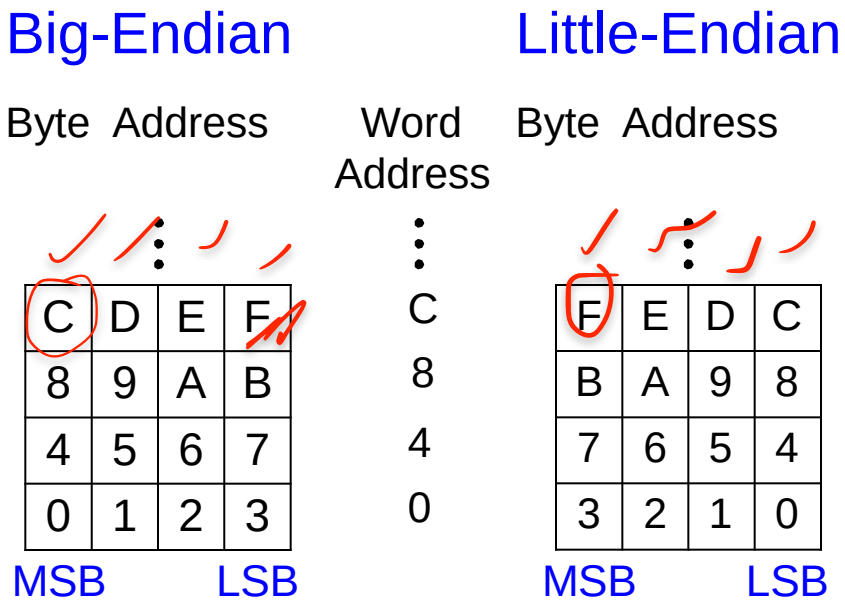
MIPS assembly code

```
sw $t7, 44($0)   # write $t7 into address 44
```



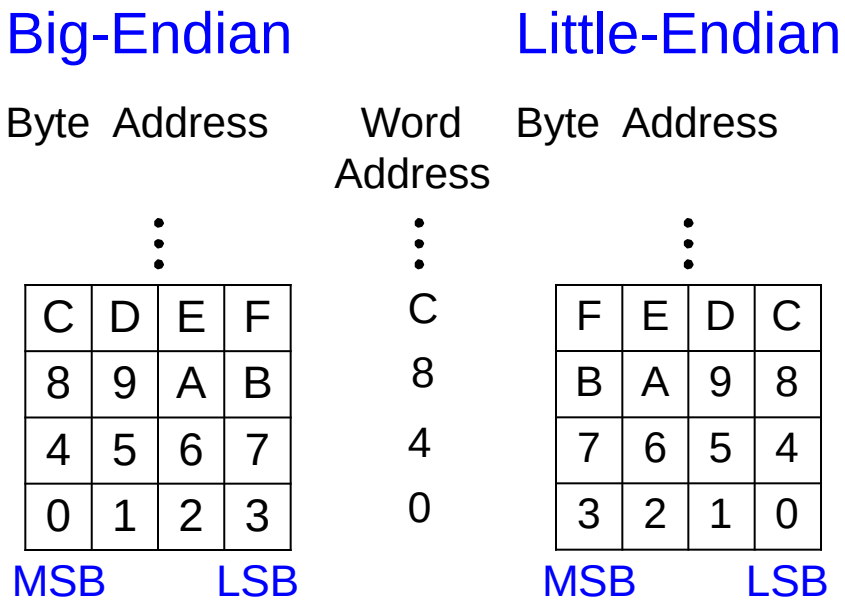
BIG-ENDIAN AND LITTLE-ENDIAN MEMORY

- How to number bytes within a word?
- Word address is the same for big- or little-endian
- Little-endian: byte numbers start at the little (least significant) end
- Big-endian: byte numbers start at the big (most significant) end



BIG-ENDIAN AND LITTLE-ENDIAN MEMORY

- From Jonathan Swift's *Gulliver's Travels* where the Little-Endians broke their eggs on the little end of the egg and the Big-Endians broke their eggs on the big end.
- As indicated by the farcical name, it doesn't really matter which addressing type is used – except when the two systems need to share data!



BIG- AND LITTLE-ENDIAN EXAMPLE

- Suppose `$t0` initially contains 0x23456789. After the following program is run on a big-endian system, what value does `$s0` contain? In a little-endian system?

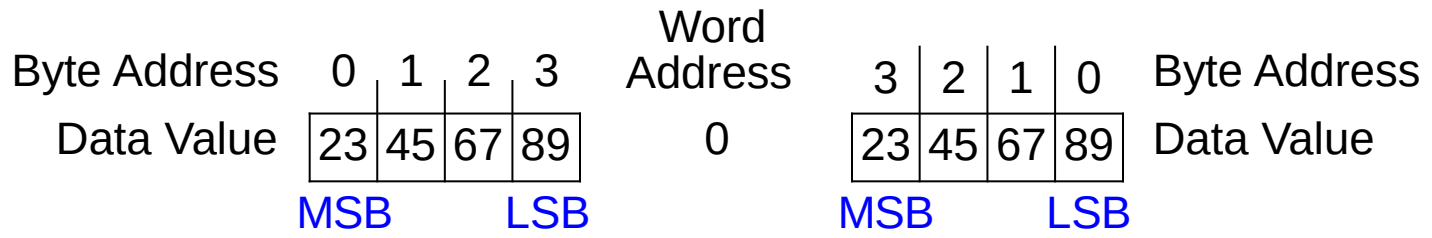
```
sw $t0, 0($0)
lb $s0, 1($0)
```

BIG- AND LITTLE-ENDIAN EXAMPLE

- Suppose `$t0` initially contains 0x23456789. After the following program is run on a big-endian system, what value does `$s0` contain? In a little-endian system?

```
sw $t0, 0($0)
lb $s0, 1($0)
```

- Big-endian: 0x00000045
- Little-endian: 0x00000067



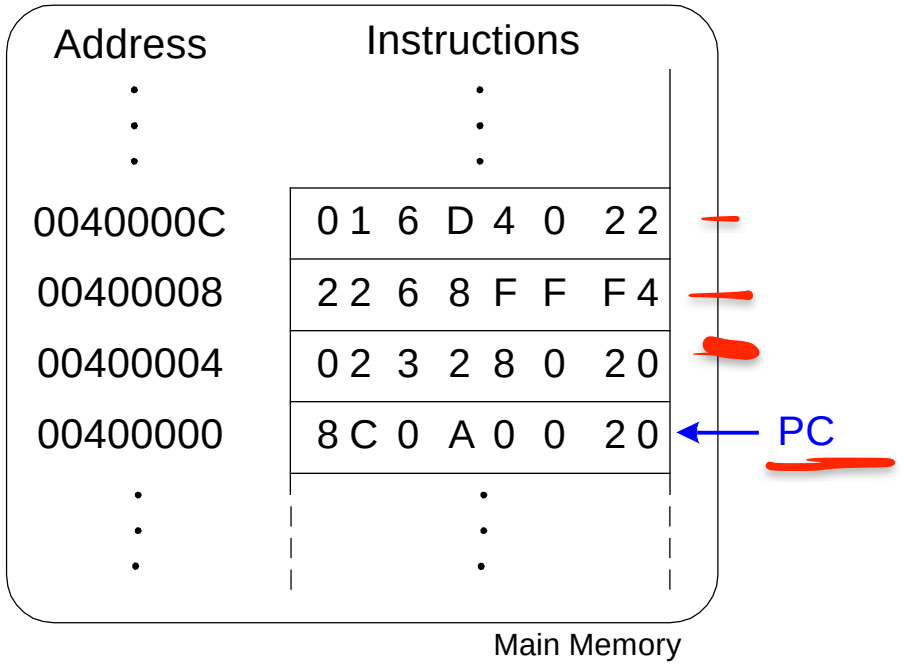
THE POWER OF THE STORED PROGRAM

- 32-bit instructions and data stored in memory
- Sequence of instructions: only difference between two applications (for example, a text editor and a video game)
- To run a new program:
  - No rewiring required
  - Simply store new program in memory
- The processor hardware executes the program:
  - *fetches* (reads) the instructions from memory in sequence
  - performs the specified operation
- The program counter (PC) keeps track of the current instruction
- In MIPS, programs typically start at memory address 0x00400000

THE STORED PROGRAM

Assembly Code	Machine Code
`lw $t2, 32($0)`	`0x8C0A0020`
`add $s0, $s1, $s2`	`0x02328020`
`addi $t0, $s3, -12`	`0x2268FFF4`
`sub $t0, $t3, $t5`	`0x016D4022`

Stored Program



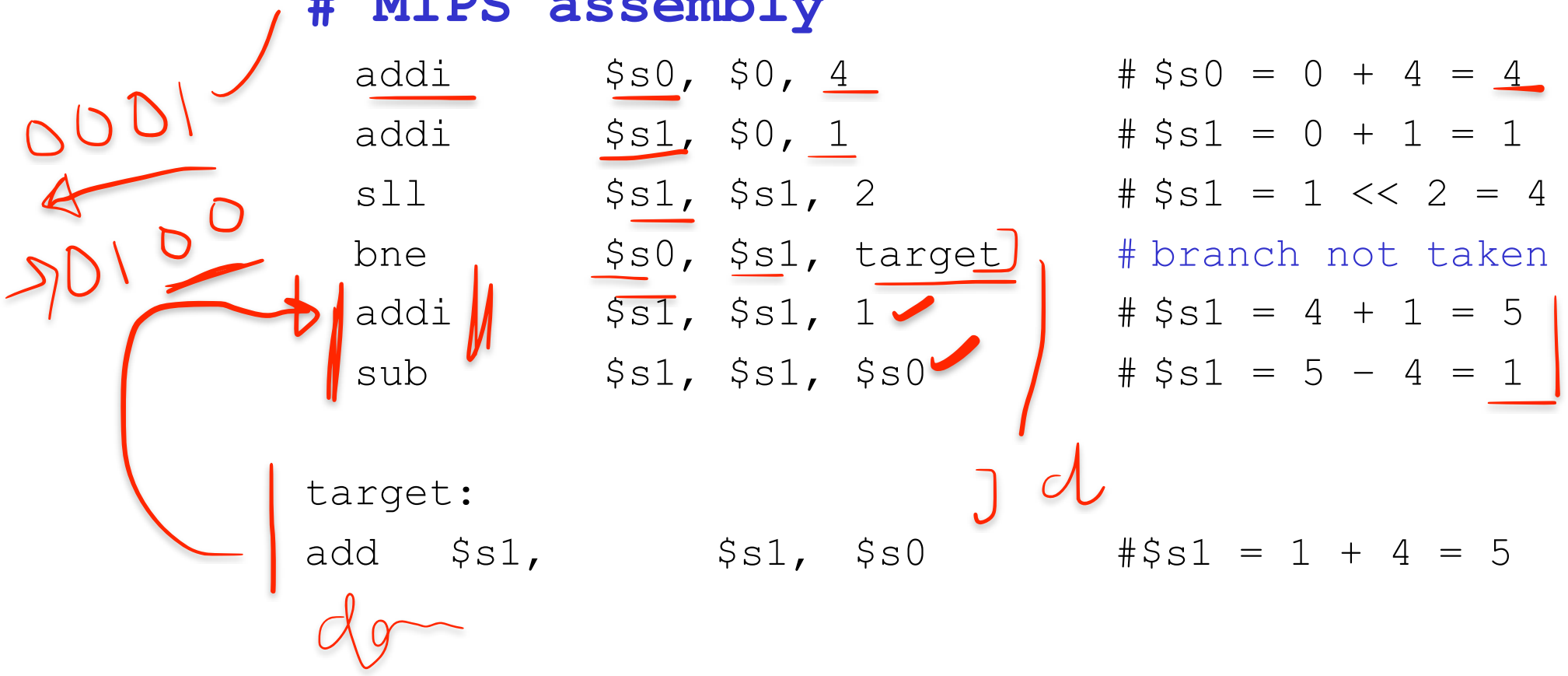
CONDITIONAL BRANCHING (BEQ)

```
# MIPS assembly
    addi $s0, $0, 4          # $s0 = 0 + 4 = 4
    addi $s1, $0, 1          # $s1 = 0 + 1 = 1
    sll  $s1, $s1, 2         # $s1 = 1 << 2 = 4
    beq  $s0, $s1, target    # branch is taken
    addi $s1, $s1, 1         # not executed
    sub  $s1, $s1, $s0       # not executed

target:                      # label
    add  $s1, $s1, $s0       # $s1 = 4 + 4 = 8
```

Labels indicate instruction locations in a program. They cannot use reserved words and must be followed by a colon (:).

THE BRANCH NOT TAKEN (BNE)



```
# MIPS assembly
    addi    $s0, $0, 4          # $s0 = 0 + 4 = 4
    addi    $s1, $0, 1          # $s1 = 0 + 1 = 1
    sll     $s1, $s1, 2         # $s1 = 1 << 2 = 4
    bne     $s0, $s1, target    # branch not taken
    addi    $s1, $s1, 1         # $s1 = 4 + 1 = 5
    sub     $s1, $s1, $s0       # $s1 = 5 - 4 = 1

target:
    add   $s1,      $s1,  $s0   #$s1 = 1 + 4 = 5
```

UNCONDITIONAL BRANCHING / JUMPING (J)

MIPS assembly

```
addi $s0, $0, 4                # $s0 = 4
addi $s1, $0, 1                # $s1 = 1
j target                       # jump to target
sra $s1, $s1,      2           # not executed
 addi      $s1, $s1, 1         # not executed
 sub       $s1, $s1, $s0       # not executed

target:
add $s1, $s1,      $s0         #$s1 = 1 + 4 = 5
```

UNCONDITIONAL BRANCHING (JR)

```
# MIPS assembly
0x00002000    addi $s0, $0, 0x2010
0x00002004    jr   $s0
0x00002008    addi $s1, $0, 1
0x0000200C    sra  $s1, $s1, 2
0x00002010    lw   $s3, 44($s1)
```

HIGH-LEVEL CODE CONSTRUCTS

- `if` statements
- `if/else` statements
- `while` loops ✓
- `for` loops ✓

IF STATEMENT

High-level code

```
if (i == j)
   f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

IF STATEMENT

High-level code

```
if (i == j)
  f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
bne $s3, $s4, L1
add $s0, $s1, $s2

L1: sub $s0, $s0, $s3
```

Notice that the assembly tests for the opposite case (`i != j`) than the test in the high-level code (`i == j`).

IF/ELSE STATEMENT

High-level code

```
if (i == j)
   f = g + h;
else
   f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

IF/ELSE STATEMENT

High-level code

```
if (i == j)
   f = g + h;
else
   f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
     bne $s3, $s4, L1
     add $s0, $s1, $s2
     j   done
L1:  sub $s0, $s0, $s3
done:
```

WHILE LOOPS

High-level code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

while (pow != 128) {
pow = pow * 2;
x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x
```

WHILE LOOPS

High-level code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x

      addi $s0, $0, 1
      add  $s1, $0, $0
      addi $t0, $0, 128
while: beq  $s0, $t0, done
      sll  $s0, $s0, 1
      addi $s1, $s1, 1
      j    while

done:
```

Notice that the assembly tests for the opposite case (`pow == 128`) than the test in the high-level code (`pow != 128`).

FOR LOOPS

The general form of a for loop is:

FOR LOOPS

The general form of a for loop is:

```
for (initialization; condition; loop operation)  loop body
```

- `initialization:` executes before the loop begins
- `condition:` is tested at the beginning of each iteration
- `loop operation:` executes at the end of each iteration
- `loop body:` executes each time the condition is met

FOR LOOPS

High-level code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
```

FOR LOOPS

High-level code

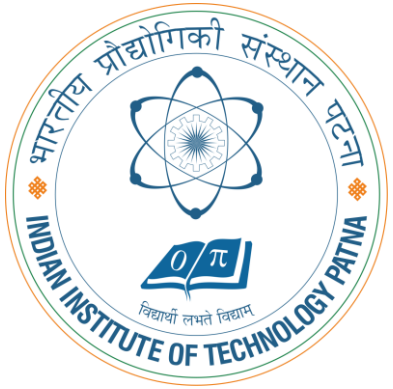
```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
  sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
      addi $s1, $0, 0
      add  $s0, $0, $0
      addi $t0, $0, 10
for:  beq  $s0, $t0, done
      add  $s1, $s1, $s0
      addi $s0, $s0, 1
      j    for
done:
```

Notice that the assembly tests for the opposite case (`i == 10`) than the test in the high-level code (`i != 10`).


भारतीय प्रौद्योगिकी संस्थान पटना
INDIAN INSTITUTE OF TECHNOLOGY PATNA
विद्यार्थी लभते विद्याम्

THANK YOU!